

Текст программы (решение варианта А для 32 варианта предметной области)**rk2_garanin.py:**

```

from operator import itemgetter

class Column:
    """Колонка данных"""
    def __init__(self, id, name, data_type, storage_size, table_id):
        self.id = id
        self.name = name
        self.data_type = data_type
        self.storage_size = storage_size
        self.table_id = table_id

class Table:
    """Таблица данных"""
    def __init__(self, id, name):
        self.id = id
        self.name = name

class ColumnTable:
    """
    Колонки таблиц для реализации связи многие-ко-многим
    """
    def __init__(self, table_id, column_id):
        self.table_id = table_id
        self.column_id = column_id

# Данные
def get_data():
    tables = [
        Table(1, 'Users'),
        Table(2, 'Orders'),
        Table(3, 'Products'),
        Table(11, 'System Users'),
        Table(22, 'System Orders'),
        Table(33, 'Archive'),
    ]

    columns = [
        Column(1, 'UserID', 'int', 100, 1),
        Column(2, 'Username', 'varchar', 250, 1),
        Column(3, 'OrderID', 'int', 150, 2),
        Column(4, 'ProductID', 'int', 120, 2),
        Column(5, 'ProductName', 'varchar', 300, 3),
        Column(6, 'Price', 'decimal', 80, 3),
    ]

    cols_tabs = [
        ColumnTable(1, 1), ColumnTable(1, 2),
        ColumnTable(2, 3), ColumnTable(2, 4),
        ColumnTable(3, 5), ColumnTable(3, 6),
        ColumnTable(11, 1), ColumnTable(11, 2),
        ColumnTable(22, 3), ColumnTable(22, 4),
        ColumnTable(33, 5),
    ]

    return tables, columns, cols_tabs

# Функции бизнес-логики

```

```

def get_one_to_many(tables, columns):
    """Соединение данных один-ко-многим"""
    return [
        (c.name, c.storage_size, t.name)
        for t in tables
        for c in columns
        if c.table_id == t.id
    ]

def get_many_to_many(tables, columns, cols_tabs):
    """Соединение данных многие-ко-многим"""
    many_to_many_temp = [
        (t.name, ct.table_id, ct.column_id)
        for t in tables
        for ct in cols_tabs
        if t.id == ct.table_id
    ]
    return [
        (c.name, c.storage_size, table_name)
        for table_name, table_id, column_id in many_to_many_temp
        for c in columns
        if c.id == column_id
    ]

def task_a1_solution(one_to_many):
    """
    Задание A1: Вывести список всех связанных элементов,
    отсортированный по названию таблицы.
    """
    return sorted(one_to_many, key=itemgetter(2))

def task_a2_solution(tables, one_to_many):
    """
    Задание A2: Вывести список таблиц с суммарным размером колонок,
    отсортированный по суммарному размеру (по убыванию).
    """
    res_unsorted = []
    for t in tables:
        t_cols = list(filter(lambda i: i[2] == t.name, one_to_many))
        if len(t_cols) > 0:
            t_sizes = [size for _, size, _ in t_cols]
            t_sizes_sum = sum(t_sizes)
            res_unsorted.append((t.name, t_sizes_sum))

    return sorted(res_unsorted, key=itemgetter(1), reverse=True)

def task_a3_solution(tables, many_to_many):
    """
    Задание A3: Вывести словарь, где ключ - имя таблицы (содержащее
    'System'),
    а значение - список имен колонок.
    """
    res = {}
    for t in tables:
        if 'System' in t.name:
            t_cols = list(filter(lambda i: i[2] == t.name, many_to_many))
            t_cols_names = [name for name, _, _ in t_cols]
            res[t.name] = t_cols_names

    return res

# Основной запуск
def main():
    tables, columns, cols_tabs = get_data()
    one_to_many = get_one_to_many(tables, columns)

```

```

many_to_many = get_many_to_many(tables, columns, cols_tabs)

print('Задание A1')
print(task_a1_solution(one_to_many))

print('\nЗадание A2')
print(task_a2_solution(tables, one_to_many))

print('\nЗадание A3')
print(task_a3_solution(tables, many_to_many))

if __name__ == '__main__':
    main()

```

test_rk2.py:

```

import unittest
from rk2_garanin import Table, Column, ColumnTable, get_one_to_many,
get_many_to_many, task_a1_solution, task_a2_solution, task_a3_solution

class TestRk2(unittest.TestCase):

    def setUp(self):
        """Инициализация тестовых данных перед каждым тестом"""
        self.tables = [
            Table(1, 'System Test'),
            Table(2, 'Alpha Table'),
            Table(3, 'Beta Table')
        ]

        self.columns = [
            Column(1, 'Col1', 'int', 10, 1), # для System Test
            Column(2, 'Col2', 'str', 20, 2), # для Alpha Table
            Column(3, 'Col3', 'int', 5, 2), # для Alpha Table
            Column(4, 'Col4', 'str', 100, 3) # для Beta Table
        ]

        self.cols_tabs = [
            ColumnTable(1, 1),
            ColumnTable(2, 2),
            ColumnTable(2, 3),
            ColumnTable(3, 4)
        ]

        # Подготовка связей
        self.one_to_many = get_one_to_many(self.tables, self.columns)
        self.many_to_many = get_many_to_many(self.tables, self.columns,
self.cols_tabs)

    def test_task_a1(self):
        """
        Тест задания A1:
        Проверяем сортировку по названию таблицы.
        """
        result = task_a1_solution(self.one_to_many)

        # Ожидаем, что первым будет 'Alpha Table' (на букву A)
        self.assertEqual(result[0][2], 'Alpha Table')
        # Вторым 'Beta Table'
        self.assertEqual(result[2][2], 'Beta Table')
        # Третьим 'System Test'
        self.assertEqual(result[3][2], 'System Test')

```

```

def test_task_a2(self):
    """
    Тест задания А2:
    Проверяем подсчет суммы (Col2(20) + Col3(5) = 25 для Alpha)
    и сортировку по убыванию размера.
    """
    result = task_a2_solution(self.tables, self.one_to_many)

    # Alpha Table имеет сумму 25, System Test - 10, Beta Table - 100.
    # Ожидаемый порядок: Beta (100), Alpha (25), System (10)

    # Проверка первого элемента (самый большой размер)
    self.assertEqual(result[0][0], 'Beta Table')
    self.assertEqual(result[0][1], 100)

    # Проверка второго элемента
    self.assertEqual(result[1][0], 'Alpha Table')
    self.assertEqual(result[1][1], 25)

def test_task_a3(self):
    """
    Тест задания А3:
    Проверяем фильтрацию таблиц со словом 'System'
    и вывод списка их колонок.
    """
    result = task_a3_solution(self.tables, self.many_to_many)

    # В тестовых данных только одна таблица содержит 'System'
    self.assertIn('System Test', result)
    self.assertNotIn('Alpha Table', result)

    # Проверяем список колонок для этой таблицы
    self.assertEqual(result['System Test'], ['Col1'])

if __name__ == '__main__':
    unittest.main()

```

Результаты выполнения:

...

Ran 3 tests in 0.000s

OK